

## Computer Assignment 03: Calculating Turbulent Boundary Layer Development

### 1. Description of the Problem

The objective of this project is to use the code developed in Project 02 for the laminar boundary layer to calculate the development of a turbulent boundary layer using the algebraic model described below. In the inner region of the boundary layer, the Reichardt model is used, and the eddy viscosity is given by

$$\nu_t = \kappa \nu \left[ \frac{yu_\tau}{\nu} - y_a^+ \tanh \left( \frac{yu_\tau}{\nu y_a^+} \right) \right] \quad (1)$$

Where  $y_a^+ = 9.7$ . In the outer region, the Clauser model is used, which assumes that the eddy viscosity reaches a constant value of  $(\nu_t)_{\max} = c_1 U_e \delta(x)$ , where  $\delta(x)$  is taken to be the location where  $u = 0.994 U_\infty$ . Whenever  $\nu_t$  calculated from Reichardt's model exceeds  $(\nu_t)_{\max}$ , the Clauser model is used, and the derivative of  $\nu_t$  is set to zero. Model constants  $\kappa = 0.41$  and  $c_1 = 0.001$  are used, and the friction velocity is found by taking the numerical derivative of  $u$  at the wall. The same dimensional properties from Project 02 are used, with  $y_{\max} = 50\delta$  and  $N = 250$ . The objectives are to: Calculate  $\delta(x)$  and  $c_f(x)$  for each  $x$  location and confirm that the  $\delta(x)$  grows like  $\delta(x) = 0.375x(Re_x)^{-1/5}$ ; obtain the apparent origin of the turbulent boundary layer  $x_{0,\text{turb}}$ ; confirm the boundary layer follows the “law of the wall”; determine the sensitivity of the model to  $\kappa$ ,  $c_1$ , and  $s$ ; discuss the necessary changes in grid resolution and stretching factor when  $Re$  is increased by a factor of 10; and discuss implementation issues.

### 2. Description of the Numerical Method

The numerical method used is nearly identical to Project 02, the Crank-Nicolson algorithm is used to march in the  $x$ -direction, a sinh-based stretched grid is used in  $\eta$ , a fourth-order central difference scheme is used for the discretization of  $\eta$  (except at the first interior points, where a second-order discretization is used), and banded linear solvers are used for both  $u^*$  and  $v^*$  at each  $x$ -step. However, for this project  $\nu^*$  and  $\frac{\partial \nu^*}{\partial \eta}$  are no longer constants ( $\nu^* = 1$ ,  $\frac{\partial \nu^*}{\partial \eta} = 0$ ), instead they are computed from the turbulence model at each  $x$ -step. Nevertheless, the coefficients  $A_j$  and  $B_j$  retain the same formula as in Project 02:

$$A_j = \frac{v_j^*}{J_j^*} - \frac{C}{J_j^{*2}} \frac{\partial \nu^*}{\partial \eta} \bigg|_j + \frac{C \nu_j^* J_{\eta,j}^*}{J_j^{*3}}, \quad B_j = \frac{-C \nu_j^*}{J_j^{*2}} \quad (2)$$

The friction velocity is nondimensionalized as  $u_\tau^* = u_\tau / U_\infty$ . From the definition  $u_\tau^2 = \nu (\partial u / \partial y)|_w$  and the chain rule, the following relationship is obtained:

$$u_\tau^{*2} = \frac{1}{Re_\delta J_0^*} \frac{\partial u^*}{\partial \eta} \bigg|_0 \quad (3)$$

Where  $Re_\delta = U_\infty \delta / \nu_\infty$ , and the wall gradient is computed via a 2nd-order forward difference

stencil:

$$\left. \frac{\partial u^*}{\partial \eta} \right|_0 = \frac{4u_1^* - u_2^*}{2\Delta\eta} \quad (4)$$

The wall coordinate is expressed as  $y^+ = y^* u_\tau^* Re_\delta$ , and the nondimensionalized effective viscosity as  $\nu_{eff}^* = 1 + \nu_t/\nu_\infty$ . The Reichardt and Clauser models can then be nondimensionalized by dividing by  $\nu_\infty$ :

$$\frac{\nu_t}{\nu_\infty} = \kappa \left[ y^+ - y_a^+ \tanh \left( \frac{y^+}{y_a^+} \right) \right], \quad \frac{\nu_{t,max}}{\nu_\infty} = c_1 Re_\delta y_{0.994}^* \quad (5)$$

Where  $y_{0.994}^*$  is the  $y^*$  location where  $u^* = 0.994$ , found by linear interpolation of the generated data. Therefore, the nondimensional effective viscosity can be expressed in the Reichardt and Clauser regions as

$$\nu_{eff}^* = 1 + \kappa \left[ y^+ - y_a^+ \tanh \left( \frac{y^+}{y_a^+} \right) \right], \quad \nu_{eff}^* = 1 + c_1 Re_\delta y_{0.994}^* \quad (6)$$

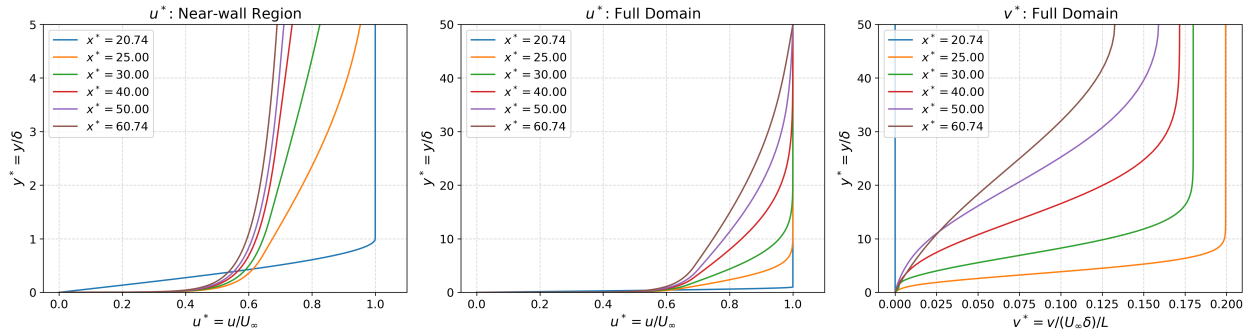
Finally, the derivative required for  $A_j$  can be derived using the chain rule for both regions:

$$\frac{\partial \nu_{eff}^*}{\partial \eta} = \begin{cases} \kappa u_\tau^* Re_\delta J^* \left[ 1 - \text{sech}^2 \left( \frac{y^+}{y_a^+} \right) \right], & \text{Reichardt region} \\ 0 & \text{Clauser region (constant } \nu_t) \end{cases} \quad (7)$$

At the wall ( $y^+ = 0$ ),  $\text{sech}^2(0) = 1$ , so  $d\nu^*/d\eta = 0$ . Furthermore, the skin friction coefficient can be expressed in terms of the nondimensional friction velocity as  $c_f = 2(u_\tau^*)^2$ . The models and logic described above can now be implemented into the program written for Project 02, the result of which is displayed in Appendix A.

### 3. Presentation of Results

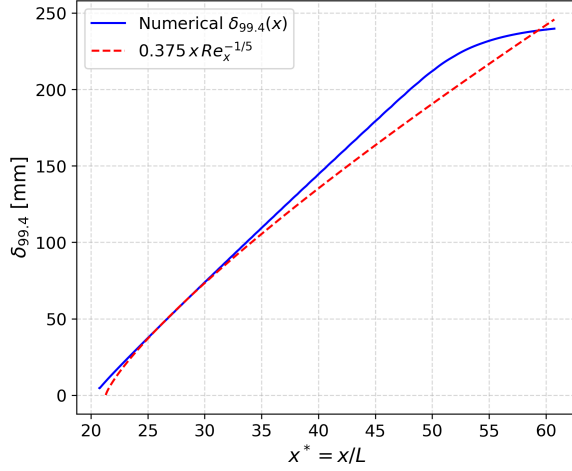
The solution is marched from  $x_0^* = 20.74$  to  $x_{max}^* = 60.74$  using  $N_x = 20,000$  steps. At the first grid point,  $y_1^+ \approx 0.07$ , confirming the point is within the viscous sublayer.



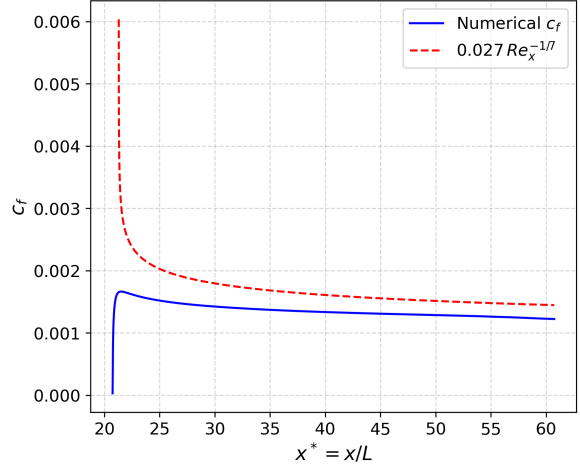
**Figure 1:** Velocity profiles at downstream locations. Left:  $u^*$  vs.  $y^*$  near the wall, showing the initial profile. Middle:  $u^*$  vs.  $y^*$  over the full domain, showing boundary layer growth. Right:  $v^*$  vs.  $y^*$  showing the wall-normal profiles.

The apparent origin of the turbulent boundary layer  $x_{0,turb} = 21.3$  m was obtained by linearizing the provided growth correlation. Raising  $\delta = C(x - x_{0,turb})^{4/5}$  to the power of 5/4 gives  $\delta^{5/4} = C^{5/4}(x - x_{0,turb})$ , which is linear in  $x$  and equals zero at  $x = x_{0,turb}$ . Following this, a linear regression was used on the downstream data (filtered with  $H < 1.4$  to omit laminar results) to obtain  $x_{0,turb}$ . Figure 2 below shows agreement between the numerical

$\delta(x)$  and the correlation over the turbulent region.



**Figure 2:** Boundary layer growth: numerical  $\delta_{99.4}(x)$  vs. empirical correlation.

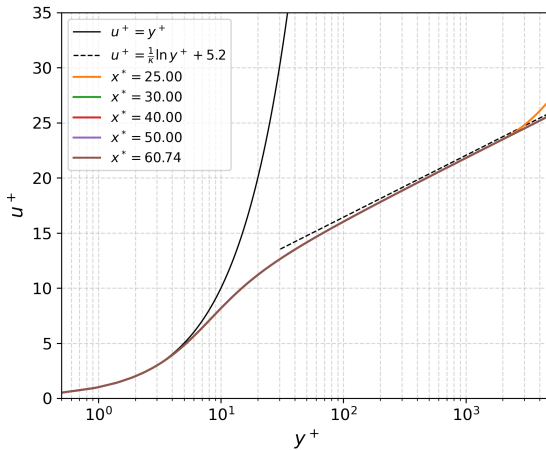


**Figure 3:** Skin friction coefficient: numerical  $c_f(x)$  vs. empirical correlation.

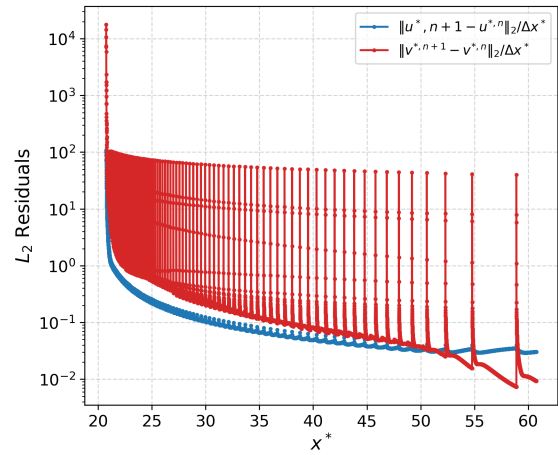
**Table 1:** Boundary layer quantities at selected streamwise locations.

| $x^*$ | $\delta_{99.4}/\delta$ | $\delta^*/\delta$ | $\theta/\delta$ | $H$   | $c_f$                  | $u_\tau^*$             |
|-------|------------------------|-------------------|-----------------|-------|------------------------|------------------------|
| 20.74 | 0.936                  | 0.375             | 0.139           | 2.692 | $2.998 \times 10^{-5}$ | $3.872 \times 10^{-3}$ |
| 25.00 | 7.480                  | 1.106             | 0.808           | 1.368 | $1.510 \times 10^{-3}$ | $2.757 \times 10^{-2}$ |
| 30.00 | 14.87                  | 2.063             | 1.541           | 1.339 | $1.424 \times 10^{-3}$ | $2.668 \times 10^{-2}$ |
| 40.00 | 28.91                  | 3.829             | 2.916           | 1.313 | $1.336 \times 10^{-3}$ | $2.584 \times 10^{-2}$ |
| 50.00 | 42.50                  | 5.477             | 4.210           | 1.301 | $1.287 \times 10^{-3}$ | $2.537 \times 10^{-2}$ |
| 60.74 | 47.96                  | 7.056             | 5.385           | 1.310 | $1.224 \times 10^{-3}$ | $2.474 \times 10^{-2}$ |

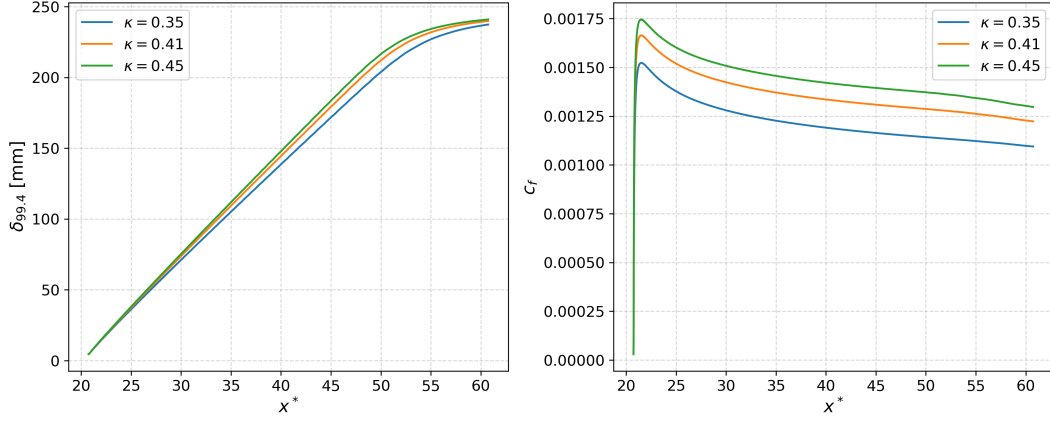
The shape factor drops rapidly from  $H \approx 2.69$  (laminar) to  $H \approx 1.3$  (turbulent) between  $x^* = 20.74$  and  $x^* = 25$ , confirming a rapid transition from the initial cubic laminar profile. The downstream values of  $H \approx 1.3$  are in agreement with the theoretical shape factor for a turbulent boundary layer on a smooth flat plate.



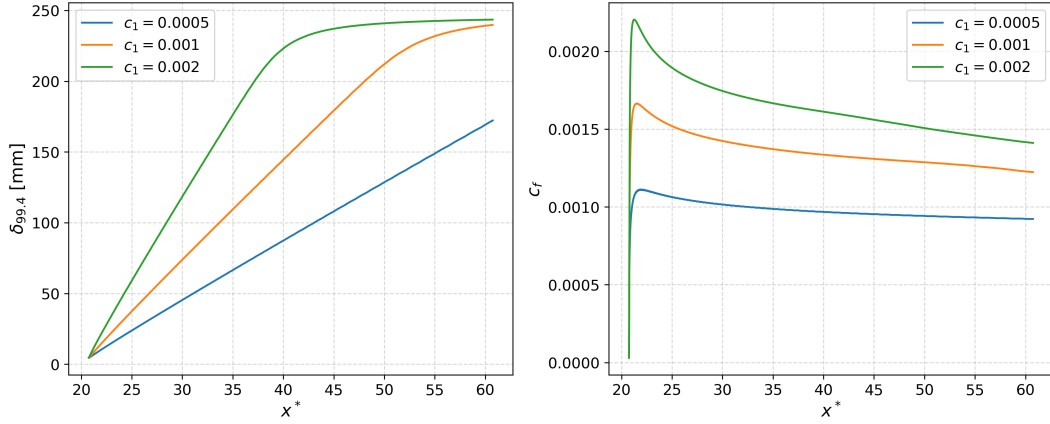
**Figure 4:** Law of the wall:  $u^+$  vs.  $y^+$  compared with  $u^+ = y^+$  and  $u^+ = (1/\kappa) \ln y^+ + 5.2$ .



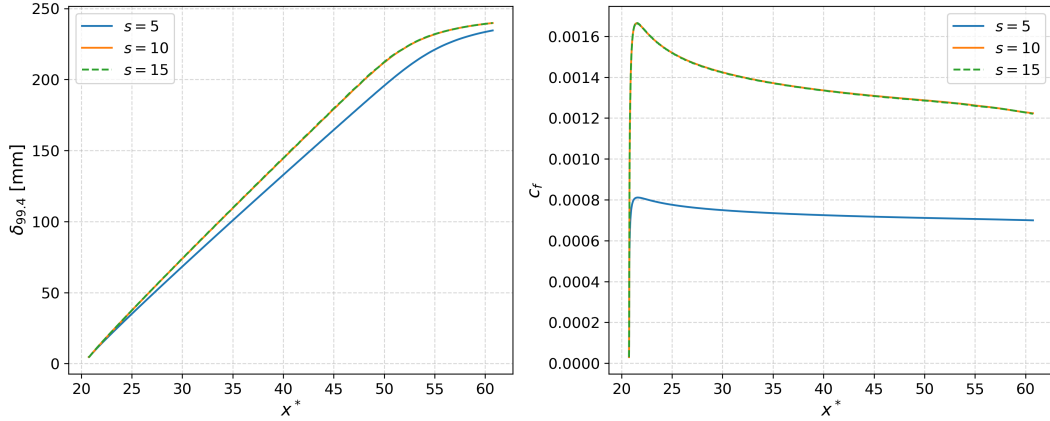
**Figure 5:**  $L_2$  residuals (normalized by  $\Delta x^*$ ) vs.  $x^*$ .



**Figure 6:** Sensitivity to  $\kappa$  ( $c_1 = 0.001$ ,  $s = 10$  fixed).



**Figure 7:** Sensitivity to  $c_1$  ( $\kappa = 0.41$ ,  $s = 10$  fixed).



**Figure 8:** Sensitivity to Stretching Factor  $s$  ( $\kappa = 0.41$ ,  $c_1 = 0.001$  fixed).

## 4. Discussion of Results

### 4.1. General description

Figure 1 shows that the initial cubic laminar profile at  $x^* = 20.74$  is confined to  $y^* \leq 1$ , while by  $x^* = 60.74$ ,  $\delta_{99.4} \approx 48\delta$  approaches  $y_{\max} = 50\delta$ . The  $v^*$  profiles confirm the satisfaction of continuity. Inside the boundary layer,  $\frac{\partial u^*}{\partial x^*} < 0$  (the boundary layer is growing), so  $\frac{\partial v^*}{\partial y^*} > 0$  (fluid is displaced away from the wall). Outside the boundary layer,  $u^* = 1$ , so

both derivatives are zero and  $v^*$  asymptotes to a constant. This constant decreases with downstream distance because  $\frac{\partial u^*}{\partial x^*}$  decreases in magnitude. The initial overlap in  $v^*$  profiles for  $x^* = 50$  and  $x^* = 60.74$  occurs because as the boundary layer grows, its edge is constrained by  $y_{\max}$ . Executing the program with  $y_{\max} = 50\delta$  and  $N = 1000$  resolved this issue.

Agreement between the simulated  $c_f$  and the empirical correlation can be observed in Figure 3. The initial disagreement is expected because the correlation assumes a fully turbulent boundary layer growing from zero thickness at  $x_{0,\text{turb}}$ . This assumption predicts a large gradient at the wall for a thin boundary layer. However, at  $x_{0,\text{turb}}$  the numerical solution is still relaxing from the laminar cubic profile, which has a smaller gradient at the wall in comparison. Figure 4 confirms that the numerical solution closely agrees with the law of the wall, with a slight deviation in the outer region for  $x^* = 25$ , as again, the solution relaxes from the initial condition.

Figures 6–8 show the sensitivity to the three model/grid parameters. At  $x^* = 40.74$ , varying  $\kappa$  by 1% produces roughly a 0.24% to 0.28% change in  $\delta$  and a 0.66% to 0.74% change in  $c_f$ , while varying  $c_1$  by 1% produces roughly a 0.5% to 0.8% change in  $\delta$  and a 0.21% to 0.55% change in  $c_f$ . Therefore,  $c_f$  is more sensitive to  $\kappa$  and  $\delta_{99.4}$  is more sensitive to  $c_1$ . This makes sense intuitively, as  $\kappa$  controls inner region viscosity/mixing (Reichardt model), where the wall gradient is determined, and  $c_1$  controls outer region viscosity/mixing (Clauser model) where increased mixing spreads the velocity transition over a wider distance, increasing  $\delta$ . For the stretching factor, the results for  $s = 10$  and  $s = 15$  are nearly identical, demonstrating grid convergence. However,  $s = 5$  differs significantly due to insufficient near-wall resolution and because its first point falls within the buffer layer ( $y^+ \approx 25.7$ ).

Increasing the Reynolds number by a factor of 10 would create a larger velocity gradient at the wall, increasing  $u_\tau$  and effectively reducing the thickness of the viscous sublayer. This would require a larger stretching factor  $s$  (and as a result, possibly more grid points  $N$  to maintain fidelity in the outer layer) to resolve the sharper gradient and to keep  $y_1^+$  within the viscous sublayer.

An implementation challenge was the  $\text{sech}^2(y^+/y_a^+)$  term in  $d\nu^*/d\eta$ , which resulted in overflow errors for large  $y^+/y_a^+$ . Since  $\text{sech}^2(x) \rightarrow 0$  for large  $x$ . Line 162 in Appendix A clamps the argument to 50 ( $\text{sech}^2(50) \approx 10^{-43}$ ), which allows the program to proceed without errors. Another implementation issue encountered was severe fluctuations in the  $v^*$  residual, which are addressed in the Accuracy and Stability section below.

## 4.2. Accuracy and stability

The  $u^*$  residual in Figure 5 decreases as  $x^*$  increases (as to be expected with the Crank-Nicolson scheme), which indicates convergence of the solution. The large initial residual is a result of the solution's rapid adjustment from the laminar initial condition. The  $u^*$  residual exhibits minuscule oscillations, while the  $v^*$  residual exhibits large oscillations. Further investigation confirmed that these oscillations (for both  $u^*$  and  $v^*$ ) occur when the Clauser cap transitions between grid points. When the boundary layer grows enough for the next grid point to leave the Clauser cap,  $A_j$  becomes non-zero, causing a step change in the solution for  $u^*$  at that step in  $x^*$ , which is amplified through the continuity equation when solving for  $v^*$ . Therefore, the residual fluctuations are an artifact of the model itself. In order to resolve this issue, one could smooth the transition between the Clauser and Reichardt models, as opposed to using a binary transition.

## A. Copy of Program Listing

```
1 # Christian DiPietrantonio
2 # ME 5311: Computational Methods to Viscous Flows
3 # Computer Assignment 03
4 # 04/30/2026
5
6 import numpy as np
7 import matplotlib
8 matplotlib.use('Agg')
9 import matplotlib.pyplot as plt
10 from scipy.linalg import solve_banded
11 import sys
12
13 # -----
14 # 1. SET PARAMETERS
15 # -----
16
17 def set_parameters():
18     """Define all physical constants, grid parameters, and nondimensional quantities."""
19
20     params = {}
21
22     # --- physical properties ---
23     params["nu_inf"] = 1.5e-6
24     params["U_inf"] = 30.0
25     params["L"] = 1.0
26     params["delta"] = 0.005
27
28     # --- grid parameters ---
29     params["N"] = 250 # number of grid points in eta (y-direction)
30     params["y_max"] = 50 * params["delta"] # final y
31     params["s"] = 10 # stretching factor
32     params["deta"] = 1.0 / (params["N"] - 1) # step size in eta
33
34     # --- turbulence model constants ---
35     params["kappa"] = 0.41
36     params["c1"] = 0.001
37     params["ya_plus"] = 9.7
38
39     # --- nondimensional quantities ---
40     params["Re_L"] = params["U_inf"] * params["L"] / params["nu_inf"]
41     params["Re_delta"] = params["U_inf"] * params["delta"] / params["nu_inf"]
42     params["C"] = (1.0 / params["Re_L"]) * (params["L"] / params["delta"])**2
43     # non-dimensional coefficient
44
45     # --- streamwise domain ---
46     params["x0"] = (params["U_inf"] * params["delta"]**2) / (params["nu_inf"]
47     * 4.91**2) # initial x from Blasius relation
48     params["x0_star"] = params["x0"] / params["L"] # non-dimensional initial x, x0
49     *
50     params["x_max"] = params["x0"] + 40 * params["L"] # final x
51     params["x_max_star"] = params["x_max"] / params["L"] # non-dimensional final x,
52     x_max*
```

```

49     params["Nx"]          = 20000    # number of steps in x*
50     params["dx_star"]     = (params["x_max_star"] - params["x0_star"]) / params["Nx"] #
        step size in x*
51
52     # -- output locations ---
53     params["x_output"]    = [params["x0_star"], 25, 30, 40, 50, params["x_max_star"]]
54
55     return params
56
57     # -----
58     # 2. COMPUTE GRID
59     # -----
60
61     def compute_grid(params):
62         """Compute the stretched grid, the Jacobian and its derivative."""
63
64         N          = params["N"]
65         y_max      = params["y_max"]
66         s          = params["s"]
67         delta      = params["delta"]
68         eta        = np.linspace(0.0, 1.0, N)
69         y_phys     = y_max * np.sinh(s * eta) / np.sinh(s)
70         y_star     = y_phys / delta
71         J_star     = (y_max * s * np.cosh(s * eta)) / (delta * np.sinh(s))
72         J_star_eta = (y_max * s**2 * np.sinh(s * eta)) / (delta * np.sinh(s))
73
74         return eta, y_phys, y_star, J_star, J_star_eta
75
76     # -----
77     # 3. COMPUTE INITIAL CONDITION
78     # -----
79
80     def compute_initial_condition(params, y_star):
81         """Compute the initial velocity profile at x0."""
82
83         N = params["N"]
84         u_star_0 = np.ones(N)
85
86         for j in range(N):
87             y_j = y_star[j]
88
89             if y_j <= 1.0:
90                 u_star_0[j] = y_j * (1.5 - 0.5 * y_j**2)
91             else:
92                 u_star_0[j] = 1.0
93
94         return u_star_0
95
96     # -----
97     # 4. COMPUTE nu* and dnu*/deta
98     # -----
99
100    def compute_nu(params, u_star, J_star):

```

```

101     """Compute the non-dimensional kinematic viscosity nu* and its derivative with
        respect to eta."""
102
103     N          = params["N"]
104     deta       = params["deta"]
105     kappa      = params["kappa"]
106     c1         = params["c1"]
107     ya_plus    = params["ya_plus"]
108     Re_delta   = params["Re_delta"]
109
110     # ----- Compute Friction Velocity -----
111     du_deta_wall = (4.0 * u_star[1] - u_star[2]) / (2.0 * deta)
112
113     u_tau_star_sq = du_deta_wall / (Re_delta * J_star[0])
114
115     # check for negative u_tau_star_sq
116     """
117     if u_tau_star_sq < 0:
118         u_tau_star_sq = 1e-15
119     """
120
121     u_tau_star = np.sqrt(u_tau_star_sq)
122
123     # ----- Compute y+ at each grid point -----
124     y_max      = params["y_max"]
125     s           = params["s"]
126     delta      = params["delta"]
127     eta_arr    = np.linspace(0.0, 1.0, N)
128     y_star_arr = (y_max / delta) * np.sinh(s * eta_arr) / np.sinh(s)
129     y_plus     = y_star_arr * u_tau_star * Re_delta
130
131     # ----- Compute Reichardt Eddy Viscosity -----
132     nu_t_star  = kappa * (y_plus - ya_plus * np.tanh(y_plus / ya_plus))
133
134     # ----- Compute Clauser Eddy Viscosity -----
135     idx_994 = np.searchsorted(u_star, 0.994)    # first index where u* > 0.994
136
137     if idx_994 <= 0:
138         y_star_994 = y_star_arr[0]                # edge case for if velocity is already
            >= 0.994
139     elif idx_994 >= N:
140         y_star_994 = y_star_arr[-1]                # edge case for if velocity never
            reaches 0.994
141     else:
142         u_below = u_star[idx_994 - 1]                # u* just below 0.994
143         u_above = u_star[idx_994]                    # u* just above 0.994
144         y_below = y_star_arr[idx_994 - 1]            # corresponding y*
145         y_above = y_star_arr[idx_994]                # corresponding y*
146         if abs(u_above - u_below) > 1e-15:          # prevent divide by zero
147             frac = (0.994 - u_below) / (u_above - u_below)
148             y_star_994 = y_below + frac * (y_above - y_below) # linearly interpolate for y
                * at u* = 0.994
149         else:
150             y_star_994 = y_below

```



```

151
152 nu_t_star_max = c1 * y_star_994 * Re_delta
153
154 # ----- Apply Clauser Condition
155 clauser_mask = nu_t_star > nu_t_star_max # true where Clauser is active
156 nu_t_star[clauser_mask] = nu_t_star_max
157
158 # ----- Nondimensional Effective Viscosity -----
159 nu_star_eff = 1 + nu_t_star
160
161 # ----- Compute Derivative of Effective Viscosity -----
162 arg = np.minimum(y_plus / ya_plus, 50) # clamp to prevent overflow errors
163 sech_sq = 1.0 / np.cosh(arg)**2
164 dnu_star_eff_deta = kappa * u_tau_star * Re_delta * J_star * (1 - sech_sq)
165
166 dnu_star_eff_deta[clauser_mask] = 0.0 # apply Clauser condition to derivative
167
168 #dnu_star_eff_deta[0] = 0.0 # not needed
169
170 crossover_idx = np.argmax(clauser_mask) # first grid point where Clauser is active
171 return nu_star_eff, dnu_star_eff_deta, u_tau_star, y_star_994, crossover_idx
172
173 # -----
174 # 5. COMPUTE COEFFICIENTS A_j and B_j
175 # -----
176
177 def compute_coefficients(params, v_star, J_star, J_star_eta, nu_star_eff,
178 dnu_star_eff_deta):
179     """Compute the coefficients A_j and B_j for the discretized system."""
180
181     C = params["C"]
182     A_j = (v_star / J_star) - ((C / J_star**2) * dnu_star_eff_deta) + ((C * nu_star_eff *
183 J_star_eta) / J_star**3)
184     B_j = -C * nu_star_eff / J_star**2
185
186     return A_j, B_j
187
188 # -----
189 # 6. COMPUTE v* FROM CONTINUITY
190 # -----
191
192 def compute_v_star(u_star_new, u_star_old, u_star_prev, params, J_star):
193     """Compute v* from the transformed/non-dimensional continuity equation
194     using a 4th-order approximation for dv*/deta and a 2nd-order approximation
195     for du*/dx* for the interior domain (j = 2 to N-3), and a 4th-order approximation
196     for dv*/deta and a 1st-order approximation for du*/dx* near the boundary/at
197     the initial step in x (i = 1, j = 1 and j = N-2)."""
198
199     N = params["N"]
200     dx_star = params["dx_star"]
201     deta = params["deta"]
202     l_diag = 2 # number of lower diagonals in banded system
203     u_diag = 1 # number of upper diagonals in banded system

```

```

203 # ----- Compute du*/dx* at each grid point -----
204 if u_star_prev is None:
205     du_dx_star = (u_star_new - u_star_old) / dx_star
206 else:
207     du_dx_star = (3.0 * u_star_new - 4.0 * u_star_old + u_star_prev) / (2.0 * dx_star)
208
209 # ----- Compute midpoint RHS values -----
210 f_mid = np.zeros(N)
211 for j in range(1, N):
212     J_mid = 0.5 * (J_star[j] + J_star[j-1])
213     du_dx_star_mid = 0.5 * (du_dx_star[j] + du_dx_star[j-1])
214     f_mid[j] = -J_mid * du_dx_star_mid
215
216 # ----- Set up banded system -----
217 # Number of unknowns = N-1, v*_0 = 0 (Wall BC)
218 # Unknowns: x[k] = v*_{k+1} for k = 0 to N-2
219 n = N - 1 # number of unknowns (v_star[1] to v_star[N-1])
220 c = 1.0 / (24.0 * deta) # stencil coefficient
221
222 # Banded matrix (l + u + 1, n) where l = 2 lower diagonals, u = 1 upper diagonal
223 # element[row, col] maps to ab[u + row - col, col]
224 ab = np.zeros((l_diag + u_diag + 1, n))
225 rhs_v_star = np.zeros(n)
226
227 # Row 0: Midpoint Rule for v*_1
228 ab[u_diag, 0] = 1.0 / deta # [row=0, col=0] -> ab[1 + 0 - 0, 0] = ab[1, 0]
229 rhs_v_star[0] = f_mid[1]
230
231 # Rows 1 to N-3 (corresponding to v*_2 to v*_{N-2}): 4th order stencil
232 for j in range(2, N-1):
233     r = j - 1 # row index in banded system
234
235     if j - 2 >= 1:
236         ab[u_diag + r - (r-2), r-2] = c # map value two columns behind main diagonal
237                                         # ab[3, r-2]: v*_{j-2}
238         # if j = 2, v*_0 = 0, so no contribution
239
240         ab[u_diag + r - (r-1), r-1] = -27 * c # map value one column behind main diagonal
241                                         # ab[2, r-1]: v*_{j-1}
242
243         ab[u_diag, r] = 27 * c # map value on main diagonal
244                               # ab[1, r]: v*_j
245
246         ab[u_diag + r - (r+1), r+1] = -c # map value one column ahead of main diagonal
247                                         # ab[0, r+1]: v*_{j+1}
248
249         rhs_v_star[r] = f_mid[j]
250
251 # Row N-2: Midpoint Rule for v*_{N-1}
252 r = N - 2

```

```

253     ab[u_diag + r - (r-1), r-1] = -1.0 / deta # map value one column behind main diagonal
          (1st lower diagonal)
254                                     # ab[2, r-1]: v*_{N-2}
255
256     ab[u_diag + r - r, r] = 1.0 / deta # map value on main diagonal
257                                     # ab[1, r]: v*_{N-1}
258
259     rhs_v_star[r] = f_mid[N-1]
260
261     # ----- Solve banded system -----
262     x = solve_banded((l_diag, u_diag), ab, rhs_v_star)
263
264     # ----- Assemble full v_star array -----
265     v_star = np.zeros(N)
266     v_star[0] = 0.0 # wall BC
267     v_star[1:] = x # interior and freestream points
268
269     return v_star
270
271 # -----
272 # 7. BUILD LINEAR SYSTEM FOR MOMENTUM EQUATION
273 # -----
274
275 def build_system(params, u_old, A_j, B_j):
276     """Build the banded matrix and RHS vector for the linear system resulting from the
          discretization
277     of the momentum equation."""
278
279     N = params["N"]
280     dx_star = params["dx_star"]
281     deta = params["deta"]
282     l_diag = 2 # number of lower diagonals in banded system
283     u_diag = 2 # number of upper diagonals in banded system
284
285     # Number of unknowns = N-2, u*_0 = 0 (Wall BC), u*_{N-1} = 1 (Freestream BC)
286     # Unknowns: x[k] = u*_{k+1} for k = 0 to N-3
287     n = N - 2 # number of unknowns (u_star[1] to u_star[N-2])
288
289     # Banded matrix (l + u + 1, n) where l = 2 lower diagonals, u = 2 upper diagonals
290     # element[row, col] maps to ab[u + row - col, col]
291     ab = np.zeros((l_diag + u_diag + 1, n))
292     rhs = np.zeros(n)
293
294     # Row 0: j=1 (near wall, 2nd order)
295     j = 1; k = 0
296     psi_1 = A_j[j] / (4.0 * deta)
297     psi_2 = B_j[j] / (2.0 * deta**2)
298     ab[2, k] = (u_old[j] / dx_star) - 2 * psi_2 # [row=0, col=0] -> ab[2 + 0 - 0, 0] =
          ab[2, 0]
299     ab[1, k + 1] = psi_1 + psi_2 # [row=0, col=1] -> ab[2 + 0 - 1, 1] =
          ab[1, 1]
300     rhs[k] = ((u_old[j]**2 / dx_star) + 2.0 * psi_2 * u_old[j]) + (-psi_1 - psi_2) *
          u_old[j+1]
301

```

```

302 # Rows 1 to N-4 (corresponding to u*_2 to u*_{N-3}): 4th order stencil
303 for j in range(2, N-2):
304     k = j - 1 # row index in banded system
305     phi_1     = A_j[j] / (24.0 * deta)
306     phi_2     = B_j[j] / (24.0 * deta**2)
307     c_jm2     = phi_1 - phi_2 # coefficient for u^{*,i+1}_{j-2}
308     c_jm1     = -8.0 * phi_1 + 16.0 * phi_2 # coefficient for u^{*,i+1}_{j-1}
309     c_j       = u_old[j] / dx_star - 30.0 * phi_2 # coefficient for u^{*,i+1}_j
310     c_jp1     = 8.0 * phi_1 + 16.0 * phi_2 # coefficient for u^{*,i+1}_{j+1}
311     c_jp2     = -phi_1 - phi_2 # coefficient for u^{*,i+1}_{j+2}
312     r_jm2     = -phi_1 + phi_2 # coefficient for u^{*,i}_{j-2}
313     r_jm1     = 8.0 * phi_1 - 16.0 * phi_2 # coefficient for u^{*,i}_{j-1}
314     r_j       = u_old[j] / dx_star + 30.0 * phi_2 # coefficient for u^{*,i}_j
315     r_jp1     = -8.0 * phi_1 - 16.0 * phi_2 # coefficient for u^{*,i}_{j+1}
316     r_jp2     = phi_1 + phi_2 # coefficient for u^{*,i}_{j+2}
317
318     if j - 2 >= 1:
319         ab[u_diag + k - (k-2), k-2] = c_jm2 # map value two columns behind main
320             diagonal (2nd lower diagonal) # ab[4, k-2]: u*_{j-2}
321
322         ab[u_diag + k - (k-1), k-1] = c_jm1 # map value one column behind main
323             diagonal (1st lower diagonal) # ab[3, k-1]: u*_{j-1}
324
325         ab[u_diag, k] = c_j # map value on main diagonal
326             # ab[2, k]: u*_j
327
328         ab[u_diag + k - (k+1), k+1] = c_jp1 # map value one column ahead of main
329             diagonal (1st upper diagonal) # ab[1, k+1]: u*_{j+1}
330
331         if j + 2 <= N - 2:
332             ab[u_diag + k - (k+2), k+2] = c_jp2 # map value two columns ahead of main
333                 diagonal (2nd upper diagonal) # ab[0, k+2]: u*_{j+2}
334
335         rhs_val = r_j * u_old[j]
336
337         if j - 2 >= 1:
338             rhs_val += r_jm2 * u_old[j-2] # contribution from u^{*,i}_{j-2}
339             rhs_val += r_jm1 * u_old[j-1] # contribution from u^{*,i}_{j-1}
340             rhs_val += r_jp1 * u_old[j+1] # contribution from u^{*,i}_{j+1}
341         if j + 2 <= N - 2:
342             rhs_val += r_jp2 * u_old[j+2] # contribution from u^{*,i}_{j+2}
343         else:
344             rhs_val += r_jp2 * 1.0 # contribution from u^{*,i}_{N-1} = 1.0 (
345                 Freestream BC)
346             rhs_val -= c_jp2 * 1.0 # contribution from u^{*,i+1}_{N-1} = 1.0 (
347                 Freestream BC)
348         rhs[k] = rhs_val
349
350 # Row N-3: j=N-2 (near freestream, 2nd order)
351 j = N - 2; k = j - 1

```

```

350     psi_1          = A_j[j] / (4.0 * deta)
351     psi_2          = B_j[j] / (2.0 * deta**2)
352     ab[3, k-1] = -psi_1 + psi_2 # [row=N-3, col=N-4] -> ab[2 + N-3 - N
    -4, N-4] = ab[3, N-4]
353     ab[2, k] = u_old[j] / dx_star - 2.0 * psi_2 # [row=N-3, col=N-3] -> ab[2 + N-3 - N
    -3, N-3] = ab[2, N-3]
354     # include contributions from  $u^{*,i+1}_{N-1}$  and  $u^{*,i}_{N-1} = 1.0$  (Freestream BC)
    in RHS
355     rhs[k] = (psi_1 - psi_2) * u_old[j-1] + (u_old[j]**2 / dx_star) + (2.0 * psi_2 *
    u_old[j]) + (-psi_1 - psi_2) * 1.0 - (psi_1 + psi_2) * 1.0
356
357     return ab, rhs, l_diag, u_diag
358
359 # -----
360 # 8. STEP IN X*
361 # -----
362
363 def step_in_x(params, u_star_0, J_star, J_star_eta):
364     """March from x0_star to x_max_star"""
365
366     N      = params["N"]
367     Nx     = params["Nx"]
368     dx_star = params["dx_star"]
369     x0_star = params["x0_star"]
370
371     u_star = u_star_0.copy()
372     v_star = np.zeros(N)
373     u_star_prev = None # No previous step initially -> use 1st
    order du/dx in compute_v_star
374
375     stride = max(1, Nx // 20000) # store solution every stride steps
376
377     store_indices = set(range(0, Nx + 1, stride)) # indices/step numbers at which to
    store solution
378
379     for x_out in params["x_output"]:
380         idx = int(round((x_out - x0_star) / dx_star)) # find step number corresponding to
    x_out
381         idx = max(0, min(idx, Nx)) # ensure idx is within bounds
382         store_indices.add(idx) # add idx to set store_indices (if not
    already present)
383
384     store_indices = sorted(store_indices) # sort indices for ordered storage
385     n_store = len(store_indices)
386
387     x_vals = np.zeros(n_store) # initialize array to store solution x-
    values
388     u_store = np.zeros((n_store, N)) # initialize array to store solution
    profiles
389     delta_store = np.zeros(n_store) # initialize array to store BL
    thicknesses
390     cf_store = np.zeros(n_store) # initialize array to store Cf
    coefficients

```

```

391 u_tau_star_store = np.zeros(n_store)           # initialize array to store
        nondimensional friction velocities
392 v_store = np.zeros((n_store, N))             # initialize array to store v* profiles
393
394 # initialize residual storage
395 x_res_list = []
396 u_res_list = []
397 v_res_list = []
398 v_star_prev_res = None
399 cross_idx_list = []
400 cross_x_list = []
401
402 store_map = {idx: pos for pos, idx in enumerate(store_indices)} # map from step
        number to storage index
403
404 # --- Store Initial Conditions ---
405 if 0 in store_map:
406     pos = store_map[0]
407     x_vals[pos] = x0_star           # store initial condition at x0_star if it's in
        the output list
408     u_store[pos, :] = u_star.copy()
409     v_store[pos, :] = v_star.copy()
410
411     # compute initial turbulent quantities
412     nu_star_eff_0, dnu_star_eff_0, u_tau_star_0, y_star_994_0, _ = compute_nu(params,
        u_star, J_star)
413     delta_store[pos] = y_star_994_0      # BL Thickness
414     cf_store[pos] = 2 * u_tau_star_0**2 # Skin Friction Coefficient
415     u_tau_star_store[pos] = u_tau_star_0 # Nondimensional Friction Velocity
416
417 # --- march in x ---
418 for n_step in range(Nx):
419     # compute effective viscosity from current velocity profile
420     nu_star_eff_n, dnu_star_eff_n, u_tau_star_n, y_star_994_n, cross_idx = compute_nu(
        params, u_star, J_star)
421     cross_idx_list.append(cross_idx)
422     cross_x_list.append(x0_star + (n_step + 1) * dx_star)
423
424     # compute coefficients A_j and B_j
425     A_j, B_j = compute_coefficients(params, v_star, J_star, J_star_eta, nu_star_eff_n,
        dnu_star_eff_n)
426
427     # build and solve the linear system for u* at the next x-step
428     ab, rhs_vec, l_diag, u_diag = build_system(params, u_star, A_j, B_j)
429
430     # check main diagonal for negative values
431     main_diag = ab[2, :] # main diagonal is at row index u_diag = 2
432     if np.any(main_diag < 0):
433         neg_indices = np.where(main_diag < 0)[0]
434         print(f" WARNING step {n_step+1}: negative main diagonal at indices {
            neg_indices}")
435
436     u_interior = solve_banded((l_diag, u_diag), ab, rhs_vec)
437

```

```

438     # assemble the new velocity profile
439     u_star_new      = np.zeros(N)
440     u_star_new[0]    = 0.0      # wall BC
441     u_star_new[1:N-1] = u_interior # interior points
442     u_star_new[N-1]  = 1.0      # freestream BC
443
444     # Compute v_star using continuity equation
445     v_star = compute_v_star(u_star_new, u_star, u_star_prev, params, J_star)
446
447     # update for next step
448     u_star_prev = u_star.copy() # store current u_star before updating for next
         iteration
449     u_star = u_star_new.copy() # update u_star for next iteration
450
451     # calculate residuals at every step, ensuring there is a previous point to compare
         with
452     if u_star_prev is not None and v_star_prev_res is not None:
453         u_res_list.append(np.linalg.norm(u_star - u_star_prev, ord=2) / dx_star)
454         v_res_list.append(np.linalg.norm(v_star - v_star_prev_res, ord=2) / dx_star)
455         x_res_list.append(x0_star + (n_step + 1) * dx_star)
456     v_star_prev_res = v_star.copy()
457
458     step = n_step + 1
459     if step in store_map:
460         pos = store_map[step] # find storage index for current step
461         x_vals[pos] = x0_star + step * dx_star # compute and store x-value for current
         step
462         u_store[pos, :] = u_star.copy() # store solution profile for current step
463         v_store[pos, :] = v_star.copy() # store v* profile for current step
464
465         # compute turbulent quantities at this step
466         nu_star_eff_s, dnu_star_eff_s, u_tau_star_s, y_star_994_s, _ = compute_nu(
             params, u_star, J_star)
467         delta_store[pos] = y_star_994_s # BL Thickness
468         cf_store[pos] = 2 * u_tau_star_s**2 # Skin Friction Coefficient
469         u_tau_star_store[pos] = u_tau_star_s # Nondimensional Friction Velocity
470
471     # if step % stride == 0:
472     if (n_step + 1) % 1000 == 0:
473         # print(f" Step {step}/{Nx}, x* = {x0_star + (step) * dx_star:.2f}")
474         print(f" Step {n_step+1}/{Nx}, x* = {x0_star + (n_step+1) * dx_star:.2f}, u*
             _tau = {u_tau_star_n:.6f}, delta_99/delta = {y_star_994_n:.4f}")
475
476     return x_vals, u_store, v_store, delta_store, cf_store, u_tau_star_store, stride,
         x_res_list, u_res_list, v_res_list, cross_x_list, cross_idx_list
477
478 # -----
479 # 9. COMPUTE BLASIUS SOLUTION
480 # -----
481
482 def compute_blasius(eta_max=10.0, n_points = 10000):
483     """Solve Blasius via RK4."""
484
485     h = eta_max / n_points # step size for RK4

```

```

486
487 eta_B = np.zeros(n_points + 1)      # values for eta
488 f_arr = np.zeros(n_points + 1)      # stream function at each eta
489 F_arr = np.zeros(n_points + 1)      # velocity profile u/U_inf = u*
490 H_arr = np.zeros(n_points + 1)      # d2f/deta2
491
492 f_arr[0] = 0.0                      # f(0) = 0 at the wall
493 F_arr[0] = 0.0                      # F(0) = 0 at the wall
494 H_arr[0] = 0.332                    # H(0) that results in F(n->infty) = 1
495
496 def rhs(f_val, F_val, H_val):
497
498     df = F_val                      # df/deta = F
499     dF = H_val                      # dF/deta = H
500     dH = -0.5 * f_val * H_val      # dH/deta = -(f/2)*H
501
502     return df, dF, dH
503
504 # Implement RK4 numerical method
505 for i in range(n_points):
506     eta_B[i] = i * h
507     fi, Fi, Hi = f_arr[i], F_arr[i], H_arr[i]
508
509     k1f, k1F, k1H = rhs(fi, Fi, Hi)
510     k2f, k2F, k2H = rhs(fi + 0.5*h*k1f, Fi + 0.5*h*k1F, Hi + 0.5*h*k1H)
511     k3f, k3F, k3H = rhs(fi + 0.5*h*k2f, Fi + 0.5*h*k2F, Hi + 0.5*h*k2H)
512     k4f, k4F, k4H = rhs(fi + h*k3f, Fi + h*k3F, Hi + h*k3H)
513
514     f_arr[i+1] = fi + (h/6.0)*(k1f + 2*k2f + 2*k3f + k4f)
515     F_arr[i+1] = Fi + (h/6.0)*(k1F + 2*k2F + 2*k3F + k4F)
516     H_arr[i+1] = Hi + (h/6.0)*(k1H + 2*k2H + 2*k3H + k4H)
517
518 eta_B[-1] = n_points * h
519
520 return eta_B, F_arr, f_arr # eta_B, u*/U_infty = F, stream function f
521
522 # -----
523 # 10. COMPUTE THICKNESSES AND SHAPE FACTOR
524 # -----
525
526 def compute_thicknesses(u_star, J_star, deta):
527     """Compute displacement thickness, momentum thickness, and shape factor"""
528     integrand_disp = (1.0 - u_star) * J_star
529     integrand_mom = u_star * (1.0 - u_star) * J_star
530
531     delta_star = np.trapezoid(integrand_disp, dx=deta) #integrate for displacement
532     # thickness using trapezoidal rule
533     theta = np.trapezoid(integrand_mom, dx=deta) #integrate for momentum
534     # thickness using trapezoidal rule
535
536     H = delta_star / theta if theta > 1e-15 else 0.0
537
538     return delta_star, theta, H

```



```

538 # -----
539 # 11. POST-PROCESSING
540 # -----
541
542 def post_process(params, x_vals, u_store, v_store, delta_store, cf_store,
543                 u_tau_star_store, eta, y_phys, y_star, J_star, stride, x_res_list, u_res_list,
544                 v_res_list, cross_x_list, cross_idx_list):
545     N = params["N"]
546     deta = params["deta"]
547     nu = params["nu_inf"]
548     U_inf = params["U_inf"]
549     L = params["L"]
550     delta = params["delta"]
551     Re_delta = params["Re_delta"]
552     kappa = params["kappa"]
553
554     print("Computing Blasius solution...")
555     eta_B, F_B, f_B = compute_blasius(eta_max=10.0, n_points = 10000)
556
557     output_indices = []
558     for x_out in params["x_output"]:
559         idx = np.argmin(np.abs(x_vals - x_out)) #take difference between desired x and
560         #stored x and return index of lowest entry
561         output_indices.append(idx)
562
563     colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:red', 'tab:purple', 'tab:brown']
564
565     # ----- Plot 01: u* and v* vs. y* -----
566     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
567
568     for i, idx in enumerate(output_indices):
569         x_now = x_vals[idx]
570         u_num = u_store[idx, :]
571         v_num = v_store[idx, :]
572
573         # Left panel: u* near-wall zoom
574         axes[0].plot(u_num, y_star, '-', color=colors[i % len(colors)], linewidth=1.5,
575                     label=fr'$x^*={x\_now:.2f}$')
576
577         # Middle panel: u* full domain
578         axes[1].plot(u_num, y_star, '-', color=colors[i % len(colors)], linewidth=1.5,
579                     label=fr'$x^*={x\_now:.2f}$')
580
581         # Right panel: v* full domain
582         axes[2].plot(v_num, y_star, '-', color=colors[i % len(colors)], linewidth=1.5,
583                     label=fr'$x^*={x\_now:.2f}$')
584
585     # Left panel formatting
586     axes[0].set_xlabel(r'$u^* = u / U\_infty$', fontsize=14)
587     axes[0].set_ylabel(r'$y^* = y / \delta$', fontsize =14)
588     axes[0].set_xlim([-0.05, 1.1])
589     axes[0].set_ylim([0.0, 5.0])
590     axes[0].set_title(r'$u^*$: Near-wall Region', fontsize=15)

```

```

585 axes[0].tick_params(labelsize=12)
586 axes[0].grid(True, linestyle='--', alpha=0.5)
587 axes[0].legend(fontsize=12, loc='upper left')
588
589 # Middle panel formatting
590 axes[1].set_xlabel(r'$u^* = u / U_{\infty}$', fontsize=14)
591 axes[1].set_ylabel(r'$y^* = y / \delta$', fontsize =14)
592 axes[1].set_xlim([-0.05, 1.1])
593 axes[1].set_ylim([0.0, y_star[-1]])
594 #axes[1].set_ylim([0.0, 60])
595 axes[1].set_title(r'$u^*$: Full Domain', fontsize=15)
596 axes[1].tick_params(labelsize=12)
597 axes[1].grid(True, linestyle='--', alpha=0.5)
598 axes[1].legend(fontsize=12, loc='upper left')
599
600 # Right panel formatting
601 axes[2].set_xlabel(r'$v^* = v / (U_{\infty} \delta) / L$', fontsize=14)
602 axes[2].set_ylabel(r'$y^* = y / \delta$', fontsize =14)
603 axes[2].set_ylim([0.0, y_star[-1]])
604 axes[2].set_title(r'$v^*$: Full Domain', fontsize=15)
605 axes[2].tick_params(labelsize=12)
606 axes[2].grid(True, linestyle='--', alpha=0.5)
607 axes[2].legend(fontsize=12, loc='upper left')
608
609 plt.tight_layout()
610 plt.savefig('velocity_profiles.png', dpi=300, bbox_inches='tight')
611 print('    ...Saved velocity_profiles.png')
612
613 # ----- Plot 02: Boundary Layer Growth -----
614 delta_994 = delta_store * delta # meters
615 x_phys = x_vals * L
616
617 #n_data = len(x_vals)
618 #i_start = n_data // 3 # skip initial laminar profile
619
620 # calculate shape factors at each output location
621 H_store = np.zeros(len(x_vals))
622 for i in range(len(x_vals)):
623     _, _, H_store[i] = compute_thicknesses(u_store[i, :], J_star, deta)
624
625 H_threshold = 1.4 # threshold for turbulent shape factor (laminar ~ 2.59, turbulent
626                  ~ 1.3)
627 turbulent_indices = np.where(H_store < H_threshold)[0] # record indices where H <
628                  threshold
629
630 if len(turbulent_indices) > 0:
631     i_start = turbulent_indices[0]
632 else:
633     i_start = 0 # if no turbulent indices, just start at beginning of the data
634
635 turb_mask = (delta_994[i_start:] > 0) & (x_phys[i_start:] > 0) # create boolean mask
636                  for turbulent indices (taking only values with positive x and delta values)
637
638

```

```

635 x_fit          = x_phys[i_start:][turb_mask]          # apply masks to filtered
        arrays
636 delta_994_fit  = delta_994[i_start:][turb_mask]
637 delta_994_54   = delta_994_fit**(5.0/4.0)
638
639 if len(x_fit) > 2:
640     coeffs      = np.polyfit(x_fit, delta_994_54, 1)
641     slope_fit    = coeffs[0]
642     x0_turb      = -coeffs[1] / slope_fit if abs(slope_fit) > 1e-15 else 0.0
643 else:
644     x0_turb      = x_phys[0]      # if not enough points for curve fit, just set x0_turb to
        initial x location
645
646 x0_turb_star = x0_turb / L
647
648 print(f'\n    Turbulent BL apparent origin:')
649 print(f'        x0_turb = {x0_turb:.2f} m (x0_turb* = {x0_turb_star:.2f})')
650
651 # compute the empirical correlation delta(x) for comparison
652 x_corr         = x_phys - x0_turb          # distance from apparent origin
653 x_corr_pos     = np.maximum(x_corr, 1e-15) # avoid negative/zero values
654 Re_x_corr      = U_inf * x_corr_pos / nu
655 delta_corr     = 0.375 * x_corr_pos * Re_x_corr**(-1.0/5.0)
656
657 plt.figure(figsize=(6, 5))
658 plt.plot(x_vals, delta_994 * 1000, 'b-', linewidth=1.5, label=r'Numerical $\delta_{99.4}(x)$')
659 plot_mask = x_corr > 0          # boolean mask so that only turbulent values are plotted
        for numerical correlations
660 plt.plot(x_vals[plot_mask], delta_corr[plot_mask] * 1000, 'r--', linewidth=1.5, label=
        =r'$0.375 \setminus x \setminus, Re_x^{-1/5}$')
661 plt.xlabel(r'$x^* = x/L$', fontsize=14)
662 plt.ylabel(r'$\delta_{99.4}(x)$ [mm]', fontsize=14)
663 plt.xticks(fontsize=12); plt.yticks(fontsize=12)
664 plt.grid(True, linestyle='--', alpha=0.5)
665 plt.legend(fontsize=12, loc='best')
666 plt.tight_layout()
667 plt.savefig('BL_growth.png', dpi=300, bbox_inches='tight')
668 print('    ...Saved BL_growth.png')
669
670 # ----- Plot 03: Skin Friction Coefficient -----
671 # Empirical turbulent flat-plate correlation: cf = 0.059 * Rex-1/5 or 0.027 * Rex-1/7
672 cf_corr        = np.zeros_like(x_vals)
673 #cf_corr[plot_mask] = 0.059 * Re_x_corr[plot_mask]**(-1.0/5.0) # only use correlation
        for turbulent data
674 cf_corr[plot_mask] = 0.027 * Re_x_corr[plot_mask]**(-1.0/7.0) # only use correlation
        for turbulent data
675
676 plt.figure(figsize=(6, 5))
677 plt.plot(x_vals, cf_store, 'b-', linewidth=1.5, label=r'Numerical $c_f$')
678 #plt.plot(x_vals[plot_mask], cf_corr[plot_mask], 'r--', linewidth=1.5, label=r'$0
        .059 \setminus, Re_x^{-1/5}$')

```

```

679 plt.plot(x_vals[plot_mask], cf_corr[plot_mask], 'r--', linewidth=1.5, label=r'$0
    .027\,Re_x^{-1/7}$')
680 plt.xlabel(r'$x^* = x/L$', fontsize=14)
681 plt.ylabel(r'$c_f$', fontsize=14)
682 plt.xticks(fontsize=12); plt.yticks(fontsize=12)
683 plt.grid(True, linestyle='--', alpha=0.5)
684 plt.legend(fontsize=12, loc='best')
685 plt.tight_layout()
686 plt.savefig('SkinFrictionCoeff.png', dpi=300, bbox_inches='tight')
687 print('    ...Saved SkinFrictionCoeff.png')
688
689 # ----- Plot 04: The Law of the Wall -----
690 #plot a selected downstream locations
691
692 plt.figure(figsize=(6, 5))
693
694 y_plus_theor = np.logspace(-1, 4, 500)
695 u_plus_visc = y_plus_theor # viscous sublayer u+ = y+
696 u_plus_log = (1.0 / kappa) * np.log(y_plus_theor) + 5.2 # log law region
697
698 plt.semilogx(y_plus_theor, u_plus_visc, 'k-', linewidth=1, label=r'$u^+ = y^+$')
699 plt.semilogx(y_plus_theor[y_plus_theor > 30], u_plus_log[y_plus_theor > 30], 'k--',
    linewidth=1, label=r'$u^+ = \frac{1}{\kappa} \ln y^+ + 5.2$')
700
701 for i, idx in enumerate(output_indices):
702     if i == 0: # skip initial condition
703         continue
704
705     x_now = x_vals[idx]
706     u_num = u_store[idx, :]
707     u_tau_star = u_tau_star_store[idx]
708
709     if u_tau_star < 1e-15: # protect against dividing by zero
710         continue
711
712     y_plus_local = y_star * Re_delta * u_tau_star
713     u_plus_local = u_num / u_tau_star
714
715     plt.semilogx(y_plus_local[1:], u_plus_local[1:], '-', color=colors[i % len(colors)]
    ], linewidth=1.5, label=fr'$x^*={x_now:.2f}$') #avoid y*=0 for log
716
717 plt.xlabel(r'$y^+$', fontsize=14)
718 plt.ylabel(r'$u^+$', fontsize=14)
719 plt.xlim([0.5, 5e3])
720 plt.ylim([0, 35])
721 plt.xticks(fontsize=12); plt.yticks(fontsize=12)
722 plt.grid(True, linestyle='--', alpha=0.5, which='both')
723 plt.legend(fontsize=10, loc='upper left')
724 plt.tight_layout()
725 plt.savefig('LawOfTheWall.png', dpi=300, bbox_inches='tight')
726 print('    ...Saved LawOfTheWall.png')
727
728 #Plot 05: L2 Residual
729 plt.figure(figsize=(6, 5))

```

```

730 plt.plot(x_res_list, u_res_list, 'o-', color='tab:blue', linewidth=1.5, markersize=2,
       label=r'$\|u^*,{n+1} - u^*,n\|_2 / \Delta x^*$')
731 plt.plot(x_res_list, v_res_list, 'o-', color='tab:red', linewidth=1.5, markersize=2,
       label=r'$\|v^*,{n+1} - v^*,n\|_2 / \Delta x^*$')
732 plt.yscale('log')
733 plt.xlabel(r'$x^*$', fontsize=14)
734 plt.ylabel('$L_2$ Residuals', fontsize=14)
735 plt.xticks(fontsize=12); plt.yticks(fontsize=12)
736 plt.grid(True, linestyle='--', alpha=0.5)
737 plt.legend(fontsize=10, loc='best')
738 plt.tight_layout()
739 plt.savefig('residuals_vs_x.png', dpi=300, bbox_inches='tight')
740 print("    ...Saved residuals_vs_x.png")
741
742 # Plot 06: L2 Residuals with crossover overlay
743 fig, ax1 = plt.subplots(figsize=(12, 5))
744
745 # left y-axis: residuals
746 ax1.plot(x_res_list, u_res_list, '-', color='tab:blue', linewidth=0.5, label=r'$u^*$
       residual')
747 ax1.plot(x_res_list, v_res_list, '-', color='tab:red', linewidth=0.5, label=r'$v^*$
       residual')
748 ax1.set_yscale('log')
749 ax1.set_xlabel(r'$x^*$', fontsize=14)
750 ax1.set_ylabel('$L_2$ Residuals', fontsize=14, color='black')
751 ax1.tick_params(labelsize=12)
752 ax1.grid(True, linestyle='--', alpha=0.3)
753 ax1.legend(fontsize=9, loc='lower left', bbox_to_anchor=(0, 1.02))
754
755 # right y-axis: crossover index
756 ax2 = ax1.twinx()
757 ax2.plot(cross_x_list, cross_idx_list, '-', color='tab:green', linewidth=1.0, label='
       Crossover index')
758 ax2.set_ylabel('Clauser crossover grid index', fontsize=14)
759 ax2.tick_params(axis='y', labelsize=12)
760 ax2.legend(fontsize=9, loc='lower right', bbox_to_anchor=(1, 1.02))
761
762 plt.title(f"s = {params['s']}", fontsize=15, loc='center')
763 plt.tight_layout()
764 plt.savefig('residuals_crossover.png', dpi=300, bbox_inches='tight')
765 print("    ...Saved residuals_crossover.png")
766
767 #Table: Thicknesses
768 print("\n" + "="*120)
769 print(f"{'x*':>10} {'x [m]':>10} {'delta_99/d':>12} {'d*/d':>12} {'th/d':>12} "
       f"{'H':>12} {'cf':>12} {'u_tau*':>12}")
770 print("="*120)
771 for i, idx in enumerate(output_indices):
772     x_now      = x_vals[idx]
773     x_dim      = x_now * L
774     u_num      = u_store[idx, :]
775     delta_star, theta, H = compute_thicknesses(u_num, J_star, deta)
776     print(f"{x_now:10.2f} {x_dim:10.2f} {delta_store[idx]:12.4f} {delta_star:12.6f} {
       theta:12.6f} {H:12.4f} {cf_store[idx]:12.4e} {u_tau_star_store[idx]:12.4e}")

```

```

778     print("="*120 )
779
780     return x0_turb, x0_turb_star
781
782     # -----
783     # 12. SENSITIVITY STUDY
784     # -----
785
786 def run_sensitivity(base_params, u_star_0, J_star, J_star_eta, y_star):
787     """
788     Run the solver with different values of kappa and c1 to assess sensitivity of the
789     turbulence model.
790     """
791
792     kappa_vals = [0.35, 0.41, 0.45]      # test values for von Karman constant
793     c1_vals    = [0.0005, 0.001, 0.002]  # test values for clausner constant
794     s_vals     = [5, 10, 15]             # test values for stretching factor
795
796     results_kappa = {}
797     results_c1    = {}
798     results_s     = {}
799
800     # --- Sensitivity to kappa (hold c1 = 0.001 fixed) ---
801     print('\n' + '=' * 120)
802     print('SENSITIVITY STUDY: Varying kappa (c1 = 0.001 fixed)')
803     print('=' * 120)
804     for kap in kappa_vals:
805         params = base_params.copy()
806         params["kappa"] = kap
807         params["c1"]    = 0.001
808         print(f'\n    Running kappa = {kap}...')
809         x_v, u_s, v_s, d_s, cf_s, ut_s, _, _, _, _ = step_in_x(params, u_star_0.copy
810             (), J_star, J_star_eta)
811         results_kappa[kap] = (x_v, d_s, cf_s)
812
813     # --- Sensitivity to c1 (hold kappa = 0.41 fixed) ---
814     print('\n' + '=' * 120)
815     print("SENSITIVITY STUDY: Varying c1 (kappa = 0.41 fixed)")
816     print('=' * 120)
817     for c1v in c1_vals:
818         params = base_params.copy()
819         params["kappa"] = 0.41
820         params["c1"]    = c1v
821         print(f'\n    Running c1 = {c1v}...')
822         x_v, u_s, v_s, d_s, cf_s, ut_s, _, _, _, _ = step_in_x(params, u_star_0.copy
823             (), J_star, J_star_eta)
824         results_c1[c1v] = (x_v, d_s, cf_s)
825
826     # --- Sensitivity to s (hold kappa = 0.41 and c1 = 0.001 fixed) ---
827     print('\n' + '=' * 120)
828     print("SENSITIVITY STUDY: Varying s (kappa = 0.41, c1 = 0.001 fixed)")
829     print('=' * 120)
830     for s_val in s_vals:
831         params = base_params.copy()

```

```

829     params["kappa"]    = 0.41
830     params["c1"]       = 0.001
831     params["s"]        = s_val
832     print(f'\n    Running s = {s_val}...')
833     _, _, y_star_s, J_star_s, J_star_eta_s = compute_grid(params)
834     u_star_0_s = compute_initial_condition(params, y_star_s)
835     x_v, u_s, v_s, d_s, cf_s, ut_s, _, _, _, _ = step_in_x(params, u_star_0_s.
        copy(), J_star_s, J_star_eta_s)
836     results_s[s_val] = (x_v, d_s, cf_s)
837
838     # --- Plot sensitivity to kappa ---
839     delta_ref = base_params["delta"]
840     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
841
842     for kap in kappa_vals:
843         x_v, d_s, cf_s = results_kappa[kap]
844         axes[0].plot(x_v, d_s * delta_ref * 1000, linewidth=1.5, label=fr'$\kappa={kap}$')
845         axes[1].plot(x_v, cf_s, linewidth=1.5, label=fr'$\kappa={kap}$')
846
847     axes[0].set_xlabel(r'$x^*$', fontsize=14)
848     axes[0].set_ylabel(r'$\delta_{99.4}$ [mm]', fontsize=14)
849     axes[0].legend(fontsize=12)
850     axes[0].grid(True, linestyle='--', alpha=0.5)
851     axes[0].tick_params(labelsize=12)
852     axes[1].set_xlabel(r'$x^*$', fontsize=14)
853     axes[1].set_ylabel(r'$c_f$', fontsize=14)
854     axes[1].legend(fontsize=12)
855     axes[1].grid(True, linestyle='--', alpha=0.5)
856     axes[1].tick_params(labelsize=12)
857     #plt.suptitle(r'Sensitivity to $\kappa$ ($c_1 = 0.001$)', fontsize=14)
858     plt.tight_layout()
859     plt.savefig('Kappa_Sensitivity.png', dpi=300, bbox_inches='tight')
860     print('\n    ...Saved Kappa_Sensitivity.png')
861
862     # --- Plot sensitivity to c1 ---
863     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
864
865     for c1v in c1_vals:
866         x_v, d_s, cf_s = results_c1[c1v]
867         axes[0].plot(x_v, d_s * delta_ref * 1000, linewidth=1.5, label=fr'$c_1={c1v}$')
868         axes[1].plot(x_v, cf_s, linewidth=1.5, label=fr'$c_1={c1v}$')
869
870     axes[0].set_xlabel(r'$x^*$', fontsize=14)
871     axes[0].set_ylabel(r'$\delta_{99.4}$ [mm]', fontsize=14)
872     axes[0].legend(fontsize=12)
873     axes[0].grid(True, linestyle='--', alpha=0.5)
874     axes[0].tick_params(labelsize=12)
875     axes[1].set_xlabel(r'$x^*$', fontsize=14)
876     axes[1].set_ylabel(r'$c_f$', fontsize=14)
877     axes[1].legend(fontsize=12)
878     axes[1].grid(True, linestyle='--', alpha=0.5)
879     axes[1].tick_params(labelsize=12)
880     #plt.suptitle(r'Sensitivity to $c_1$ ($\kappa = 0.41$)', fontsize=14)
881     plt.tight_layout()

```

```

882 plt.savefig('c1_Sensitivity.png', dpi=300, bbox_inches='tight')
883 print('\n    ...Saved c1_Sensitivity.png')
884
885 # --- Plot sensitivity to s ---
886 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
887 s_styles = ['-', '--', '---']
888 for style, s_val in enumerate(s_vals):
889     x_v, d_s, cf_s = results_s[s_val]
890     axes[0].plot(x_v, d_s * delta_ref * 1000, linestyle=s_styles[style], linewidth
891                 =1.5, label=fr'$s={s_val}$')
892     axes[1].plot(x_v, cf_s, linestyle=s_styles[style], linewidth=1.5, label=fr'$s={
893                 s_val}$')
894
895 axes[0].set_xlabel(r'$x^*$', fontsize=14)
896 axes[0].set_ylabel(r'$\delta_{99.4}$ [mm]', fontsize=14)
897 axes[0].legend(fontsize=12)
898 axes[0].grid(True, linestyle='--', alpha=0.5)
899 axes[0].tick_params(labelsize=12)
900 axes[1].set_xlabel(r'$x^*$', fontsize=14)
901 axes[1].set_ylabel(r'$c_f$', fontsize=14)
902 axes[1].legend(fontsize=12)
903 axes[1].grid(True, linestyle='--', alpha=0.5)
904 axes[1].tick_params(labelsize=12)
905 #plt.suptitle(r'Sensitivity to $s$ ($\kappa = 0.41$, $c_1 = 0.001$)', fontsize=14)
906 plt.tight_layout()
907 plt.savefig('s_Sensitivity.png', dpi=300, bbox_inches='tight')
908 print('\n    ...Saved s_Sensitivity.png')
909
910 # -----
911 # 12. MAIN DRIVER
912 # -----
913 def main():
914     run_sensitivity_flag = '--no-sensitivity' not in sys.argv
915     print("=" * 120)
916     print("    ME 5311 - Project 03: Turbulent Boundary Layer Solver")
917     print("    Reichardt (inner region) + Clauser Model (outer region)")
918     print("=" * 120)
919
920     # --- Step 1: Set Parameters ---
921     print("\n[1] Setting parameters...")
922     params = set_parameters()
923     print(f"    Re_L      = {params['Re_L']:.2e}")
924     print(f"    Re_delta  = {params['Re_delta']:.2e}")
925     print(f"    x0*       = {params['x0_star']:.2f}")
926     print(f"    N         = {params['N']}")
927     print(f"    y_max     = {params['y_max']} m    ({params['y_max']/params['delta']:.0f}
928           delta)")
929     print(f"    kappa     = {params['kappa']}")
930     print(f"    c1        = {params['c1']}")
931     print(f"    ya+       = {params['ya_plus']}")
932     print(f"    Nx        = {params['Nx']},    dx* = {params['dx_star']:6f}")
933     print(f"    s         = {params['s']}")
934
935     # --- Step 2: Compute stretched grid ---

```



```

933 print("\n[2] Computing stretched grid...")
934 eta, y_phys, y_star, J_star, J_star_eta = compute_grid(params)
935 print(f"    dy_min = {y_phys[1]-y_phys[0]:.2e} m, dy_max = {y_phys[-1]-y_phys[-2]:.2e}
    m")
936 print(f"    y*_min = {y_star[1]:.4f},    y*_max = {y_star[-1]:.4f}")
937
938 #check if first point is within viscous sublayer
939 u_star_0    = compute_initial_condition(params, y_star)
940 du_deta_0   = (4.0 * u_star_0[1] - u_star_0[2]) / (2.0 * params["deta"])
941 u_tau_init  = np.sqrt(max(du_deta_0 / (params['Re_delta'] * J_star[0]), 1e-15)) #
    prevent negative numbers
942 #u_tau_init  = np.sqrt(du_deta_0 / (params["Re_delta"] * J_star[0]))
943 y_plus_init  = y_star[1] * params["Re_delta"] * u_tau_init
944 print(f"    Initial u_tau* ~ {u_tau_init:.6f}, y+_1 ~ {y_plus_init:.3f}")
945
946 # --- Step 3: Set Initial Conditions ---
947 print("\n[3] Setting initial conditions...")
948 #u_star_0 = compute_initial_condition(params, y_star)
949
950 # --- Step 4: March in x ---
951 print("\n[4] Marching in x...")
952 x_vals, u_store, v_store, delta_store, cf_store, u_tau_star_store, stride, x_res_list
    , u_res_list, v_res_list, cross_x_list, cross_idx_list = step_in_x(params,
    u_star_0, J_star, J_star_eta)
953
954 # --- Step 5: Post Process ---
955 print("\n[5] Post-processing...")
956 x0_turb, x0_turb_star = post_process(params, x_vals, u_store, v_store, delta_store,
    cf_store, u_tau_star_store, eta, y_phys, y_star, J_star, stride, x_res_list,
    u_res_list, v_res_list, cross_x_list, cross_idx_list)
957
958 # --- Step 6: Sensitivity Study ---
959 if run_sensitivity_flag:
960     print("\n[6] Running sensitivity study ...")
961     run_sensitivity(params, u_star_0, J_star, J_star_eta, y_star)
962 else:
963     print("\n[6] Skipping sensitivity study due to --no-sensitivity flag")
964
965 print("\n" + "=" * 120)
966 print("\nDone!")
967 print("=" * 120)
968
969 if __name__ == "__main__":
970     main()

```